

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1:

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

if  $n \leq 0$  then
  | return 0
else
  |  $x = 1$ 
  | for  $i = 1, \dots, n$  do
  |   |  $x = 2x$ 
  |   | MYSTERY2( $x$ )
  | return MYSTERY1( $n/2$ )+MYSTERY1( $n/2$ )+MYSTERY1( $n/2$ )+MYSTERY1( $n/2$ )+ $n^4$ + $\log_2 n$ 

```

```

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  | return 1
else
  | return MYSTERY2( $n/2$ ) +  $n^2$  +  $n$  + 1

```

Soluzione. Analizziamo innanzitutto il costo di MYSTERY2, che esegue una chiamata ricorsiva su un input pari alla metà del valore originale. Le restanti operazioni eseguite da MYSTERY2 hanno costo costante. L'equazione di ricorrenza associata a MYSTERY2 è:

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ T'(n/2) + 1 & n > 1 \end{cases}$$

Tale ricorrenza può essere risolta mediante il Master Theorem:

$$\alpha = \log_2 1 = 0 = \beta \Rightarrow T'(n) = \Theta(n^\alpha \log n) = \Theta(\log n)$$

La funzione MYSTERY1 esegue quattro chiamate ricorsive su un input ridotto di un fattore $1/2$. Il costo delle restanti operazioni è dominato dal ciclo **for**. Tale ciclo viene eseguito n volte, per valori di i compresi tra 1 ed n .

Ad ogni iterazione viene invocata la funzione MYSTERY2 con input x , inizialmente pari a 2 e successivamente raddoppiato ad ogni iterazione. Poiché il costo di MYSTERY2 è logaritmico rispetto al proprio input, il costo complessivo di questa sequenza di chiamate è:

$$\Theta(\log 2) + \Theta(\log 2^2) + \dots + \Theta(\log 2^n) = \Theta\left(\sum_{i=1}^n \log 2^i\right) = \Theta\left(\log 2 \sum_{i=1}^n i\right) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2)$$

Le altre operazioni eseguite da MYSTERY1 hanno costo asintoticamente inferiore a quello del ciclo **for**. Pertanto, l'equazione di ricorrenza associata a MYSTERY1 è:

$$T(n) = \begin{cases} 1 & n \leq 0 \\ 4T(n/2) + n^2 & n > 0 \end{cases}$$

Tale ricorrenza può essere risolta mediante il Master Theorem:

$$\alpha = \log_2 4 = 2 = \beta \Rightarrow T(n) = \Theta(n^\alpha \log n) = \Theta(n^2 \log n)$$

Pertanto, la complessità dell'algoritmo MYSTERY1 è $\Theta(n^2 \log n)$.

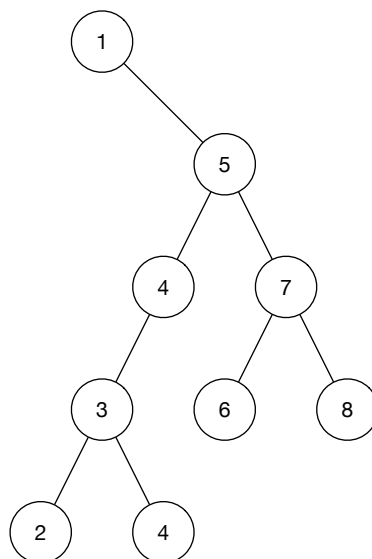
2. Considerare le seguenti sequenze di valori chiave ottenute visitando un albero binario di ricerca:

- pre-ordine: 1 5 4 3 2 4 7 6 8
- post-ordine: 2 4 3 4 6 8 7 5 1

Disegnare un albero binario di ricerca compatibile con tali visite (ce ne potrebbe essere più di uno).

Soluzione.

- Da entrambe le visite sappiamo che il nodo radice è 1 (il primo nodo della visita in pre-ordine e l'ultimo nodo della visita in post-ordine).
- Dalla visita in pre-ordine deduciamo che la radice non ha un figlio sinistro, poiché $1 < 5$.
- Dalla visita in post-ordine confermiamo che 5 è il figlio destro della radice.
- Possiamo ora concentrarci sul sottoalbero radicato in 5 (l'unico figlio della radice):
 - pre-ordine: 5 4 3 2 4 7 6 8
 - post-ordine: 2 4 3 4 6 8 7 5
- Dalla visita in post-ordine identifichiamo 7 come figlio destro di 5 (poiché $5 < 7$), mentre dalla visita in pre-ordine identifichiamo 4 come figlio sinistro di 5 (poiché $5 > 4$).
 - I valori rimanenti possono ora essere facilmente collocati nel sottoalbero sinistro o destro di 5, poiché sono tutti strettamente minori di 5 (3, 2, 4) oppure tutti strettamente maggiori di 5 (6, 8).
 - Il sottoalbero sinistro di 5 è identificato dalle seguenti visite:
 - * pre-ordine: 4 3 2 4
 - * post-ordine: 2 4 3 4
 che determinano un unico possibile sottoalbero.
 - Il sottoalbero destro di 5 è identificato dalle seguenti visite:
 - * pre-ordine: 7 6 8
 - * post-ordine: 6 8 7
 che, ancora una volta, determinano un unico possibile sottoalbero.
- In conclusione, esiste un'unica soluzione:



3. Progettare un algoritmo che riceve in input un array $A[1..n]$ di numeri naturali e due naturali min e max , e restituisce *true* se esiste un insieme di indici $I \subseteq \{1, \dots, n\}$ tale che $min \leq \sum_{i \in I} A[i] \leq max$, altrimenti restituisce *false*.

Soluzione. Si osserva che se $min = 0$ il problema è banalmente risolvibile in quanto è sufficiente scegliere $I = \emptyset$ (che produce sommatoria uguale a 0). Se invece $min > 0$, si può utilizzare la programmazione dinamica considerando i problemi $P(s, t)$, con $s \in \{1, \dots, n\}$ e $t \in \{1, \dots, min\}$, che consistono nel trovare il minimo numero maggiore o uguale a t che si può ottenere sommando valori scelti tra i primi s elementi dell'array, ovvero scelti da $A[1..s]$. Se tutti i numeri in $A[1..s]$ non permettono di raggiungere t , indicheremo $P(s, t) = \infty$. Tali problemi possono essere risolti procedendo induttivamente rispetto al parametro s . Nel caso base, $s = 1$, abbiamo:

$$P(1, t) = \begin{cases} A[1] & \text{se } A[1] \geq t \\ \infty & \text{altrimenti} \end{cases}$$

mentre per $s > 1$:

$$P(s, t) = \begin{cases} \min(P(s-1, t), A[s]) & \text{se } A[s] \geq t \\ \min(P(s-1, t), A[s] + P(s-1, t - A[s])) & \text{altrimenti} \end{cases}$$

Una volta risolti tutti questi sottoproblemi, sarà sufficiente restituire *true* se $P(n, min) \leq max$, che vuol dire che il numero minimo, maggiore o uguale di min che si può ottenere prendendo elementi dall'array $A[1..n]$ risulta essere anche minore o uguale a max . L'algoritmo 2 risolve i sottoproblemi $P(s, t)$ salvando le soluzioni nella tabella $S[1..n, 1..min]$. Al termine, restituisce il risultato del controllo $S[n, min] \leq max$.

Il costo computazionale di tale algoritmo nel caso ottimo, che si verifica quando $min = 0$, risulta essere costante. In ogni altro caso (ovvero $min \neq 0$), il costo computazionale risulta essere $T(n, min) = O(n \times min)$ in quanto si rende necessario riempire tutte le celle della tabella $S[1..n, 1..min]$ e ogni riempimento richiede tempo costante.

4. Progettare un algoritmo che dati in input un grafo $G = (V, E, w)$ orientato e pesato con pesi non negativi, un vertice $v \in V$ ed un numero non negativo D , produce in output il numero di vertici $v' \in V$ tali che esiste un cammino da v a v' con costo complessivo minore o uguale a D . Si ricorda che il costo complessivo di un cammino è la somma dei pesi degli archi del cammino. Inoltre, tenete in considerazione che il vertice di partenza v deve essere sempre conteggiato, per ogni possibile valore D in input, in quanto v è raggiungibile da se stesso con un cammino privo di archi, quindi di costo complessivo uguale a 0.

Soluzione. Per risolvere il problema è possibile utilizzare una versione adattata dell'algoritmo di Dijkstra a partire dal vertice v . Tale algoritmo visita i vertici in ordine di distanza dal vertice di partenza; è quindi possibile contare i vertici che vengono visitati fino a che non se ne visita uno a distanza superiore a D .

Algorithm 2: SUBSETINTERVAL(*Natural* $A[1..n]$, *Natural* min , *Natural* max) $\rightarrow$ *Boolean*

```

if  $min == 0$  then
   $\perp$  return true
Number  $S[1..n, 1..min]$ 
for  $t = 1$  to  $min$  do
  if  $A[1] \geq t$  then
     $S[1, t] = A[1]$ 
  else
     $S[1, t] = \infty$ 
for  $s = 2$  to  $n$  do
  for  $t = 1$  to  $min$  do
    if  $A[s] \geq t$  then
       $S[s, t] = \min(S[s - 1, t], A[s])$ 
    else
       $S[s, t] = \min(S[s - 1, t], A[s] + S[s - 1, t - A[s]])$ 
return  $S[n, min] \leq max$ 

```

Tale algoritmo viene presentato in pseudocodice come Algoritmo 3, dove viene usata la variabile ausiliaria *numVicini* per conteggiare il numero di vertici visitati prima di raggiungere vertici a distanza superiore a D , e si utilizza l'array $K[1..|V|]$ per memorizzare le distanze presunte dei vertici. L'algoritmo assume che i vertici corrispondano con numeri interi compresi fra 1 e $|V|$ in modo tale da poterli usare come indici dell'array K .

Nel caso pessimo, che si verifica quando tutti i vertici sono a distanza minore o uguale a D (quindi si deve eseguire l'algoritmo di Dijkstra sull'intero grafo), il costo computazionale risulta corrispondere con il costo computazionale dell'algoritmo di Dijkstra, ovvero $O(m \log n)$, con $m = |E|$ e $n = |V|$.

Algorithm 3: $\text{CONTA VICINI}(\text{GRAPH } G = (V, E, w), \text{ VERTEX } v, \text{ NUMBER } D) \rightarrow \text{NUMBER}$

```

/* calcola e restituisce il numero di vicini (vertici a distanza minore o uguale a D) */
NUMBER numVicini  $\leftarrow$  0
NUMBER K[1..|V|]
for  $i \leftarrow 1$  to  $n$  do
     $K[i] \leftarrow \infty$ 
 $K[v] \leftarrow 0$ 
MINPRIORITYQUEUE[INT, NUMBER] Q  $\leftarrow$  new MINPRIORITYQUEUE[INT, NUMBER]()
Q.insert( $v, K[v]$ )
while not Q.isEmpty() do
     $u \leftarrow Q.findMin()$ 
    if  $K[u] > D$  then
        /* arrivati a vertice troppo lontano, si può interrompere l'algoritmo */
        return numVicini
    else
        /* trovato un altro vicino */
        numVicini  $\leftarrow$  numVicini + 1
    Q.deleteMin()
    for  $v \in u.adjacent()$  do
        if  $K[v] = \infty$  then
            /* prima volta che si incontra v */
             $K[v] \leftarrow K[u] + w(u, v)$ 
            Q.insert( $v, K[v]$ )
        else if  $K[u] + w(u, v) < K[v]$  then
            /* scoperta di un cammino migliore per raggiungere v */
             $K[v] \leftarrow K[u] + w(u, v)$ 
            Q.decreaseKey( $v, K[v]$ )
return numVicini

```
